

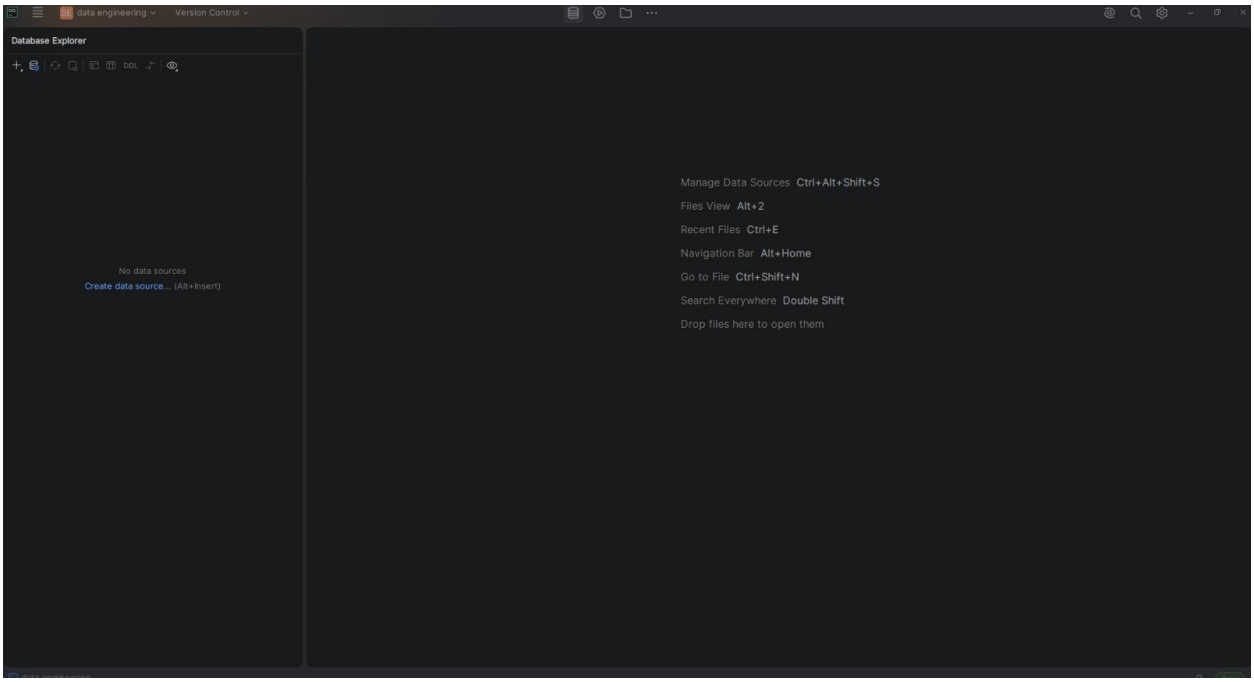
Homework 1

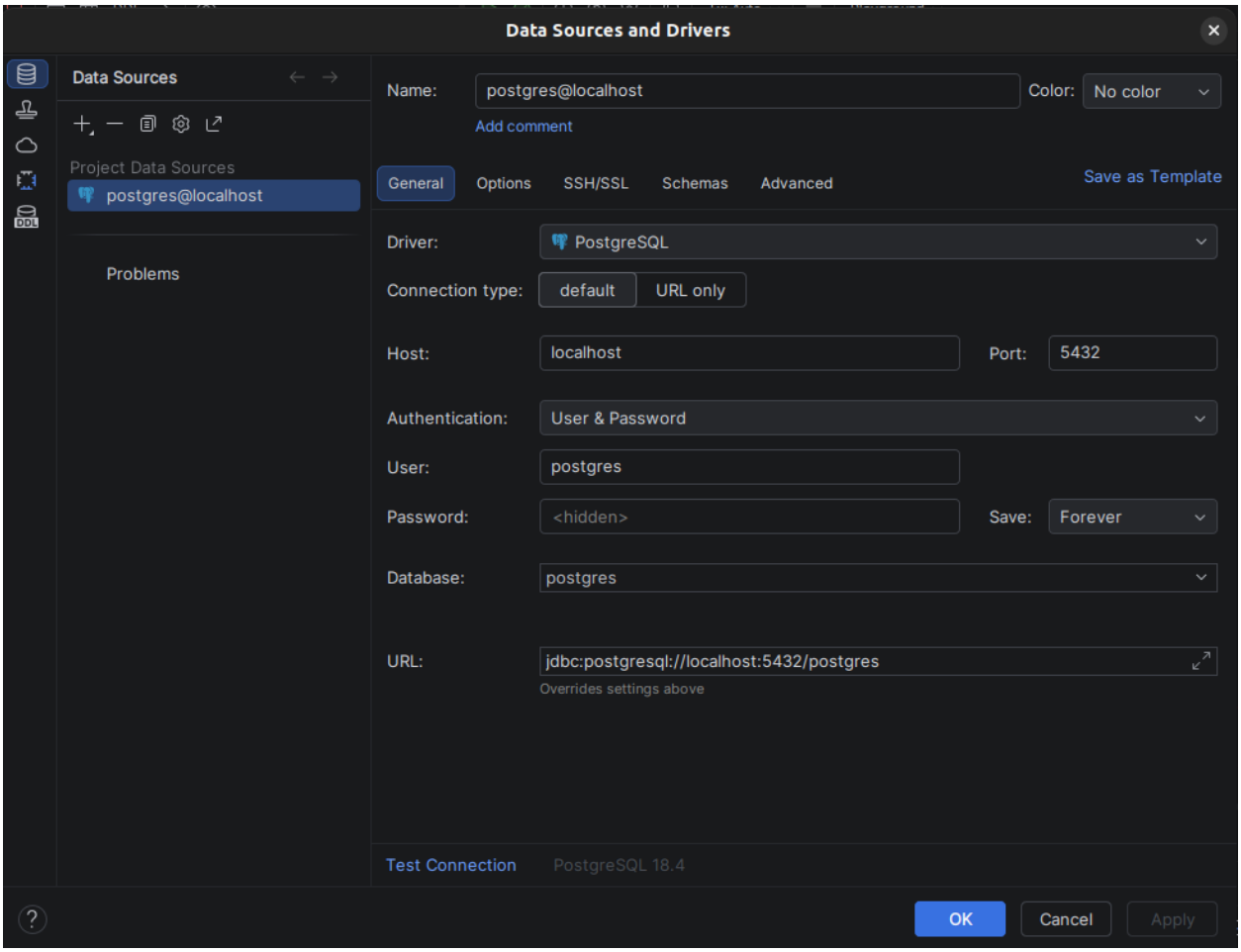
Zero to Data Engineer Specialization

Open DataGrip

Click **New Project** (or if already open):

Database Explorer → + → Data Source → PostgreSQL





Field	Value
Host	`localhost`
Port	`5432`
User	`postgres`
Password	the password you set using `\password postgres`
Database	`postgres`

```
-- ====================================================
-- Create Schema
-- ====================================================

CREATE SCHEMA IF NOT EXISTS raw;

-- ====================================================
-- Users
-- ====================================================

CREATE TABLE IF NOT EXISTS raw.users (
    user_id     INTEGER PRIMARY KEY,
    username    VARCHAR(100),
    email       VARCHAR(255),
    join_date   TIMESTAMP
);

-- ====================================================
-- Posts
-- ====================================================

CREATE TABLE IF NOT EXISTS raw.posts (
    post_id     INTEGER PRIMARY KEY,
    user_id     INTEGER,
    content     TEXT,
    created_at   TIMESTAMP,

    CONSTRAINT fk_posts_user
        FOREIGN KEY (user_id)
        REFERENCES raw.users(user_id)
);

-- ====================================================
-- Comments
-- ====================================================

CREATE TABLE IF NOT EXISTS raw.comments (
    comment_id    INTEGER PRIMARY KEY,
    post_id       INTEGER,
    user_id       INTEGER,
```

```
    text         TEXT,
    created_at   TIMESTAMP,

    CONSTRAINT fk_comments_post
        FOREIGN KEY (post_id)
        REFERENCES raw.posts(post_id),

    CONSTRAINT fk_comments_user
        FOREIGN KEY (user_id)
        REFERENCES raw.users(user_id)
);

-- ====================================================
-- Likes
-- ====================================================

CREATE TABLE IF NOT EXISTS raw.likes (
    like_id      INTEGER PRIMARY KEY,
    post_id      INTEGER,
    user_id      INTEGER,
    created_at   TIMESTAMP,

    CONSTRAINT fk_likes_post
        FOREIGN KEY (post_id)
        REFERENCES raw.posts(post_id),

    CONSTRAINT fk_likes_user
        FOREIGN KEY (user_id)
        REFERENCES raw.users(user_id)
);

-- ====================================================
-- Shares
-- ====================================================

CREATE TABLE IF NOT EXISTS raw.shares (
    share_id     INTEGER PRIMARY KEY,
    post_id      INTEGER,
    user_id      INTEGER,
    created_at   TIMESTAMP,

    CONSTRAINT fk_shares_post
        FOREIGN KEY (post_id)
        REFERENCES raw.posts(post_id),
```

```
    CONSTRAINT fk_shares_user
        FOREIGN KEY (user_id)
        REFERENCES raw.users(user_id)
);

-- ====================================================
-- Advertisements
-- ====================================================

CREATE TABLE IF NOT EXISTS raw.advertisements (
    ad_id         INTEGER PRIMARY KEY,
    user_id       INTEGER,
    title         VARCHAR(255),
    create_date   TIMESTAMP,
    cost          NUMERIC(10,2),

    CONSTRAINT fk_ads_user
        FOREIGN KEY (user_id)
        REFERENCES raw.users(user_id)
);

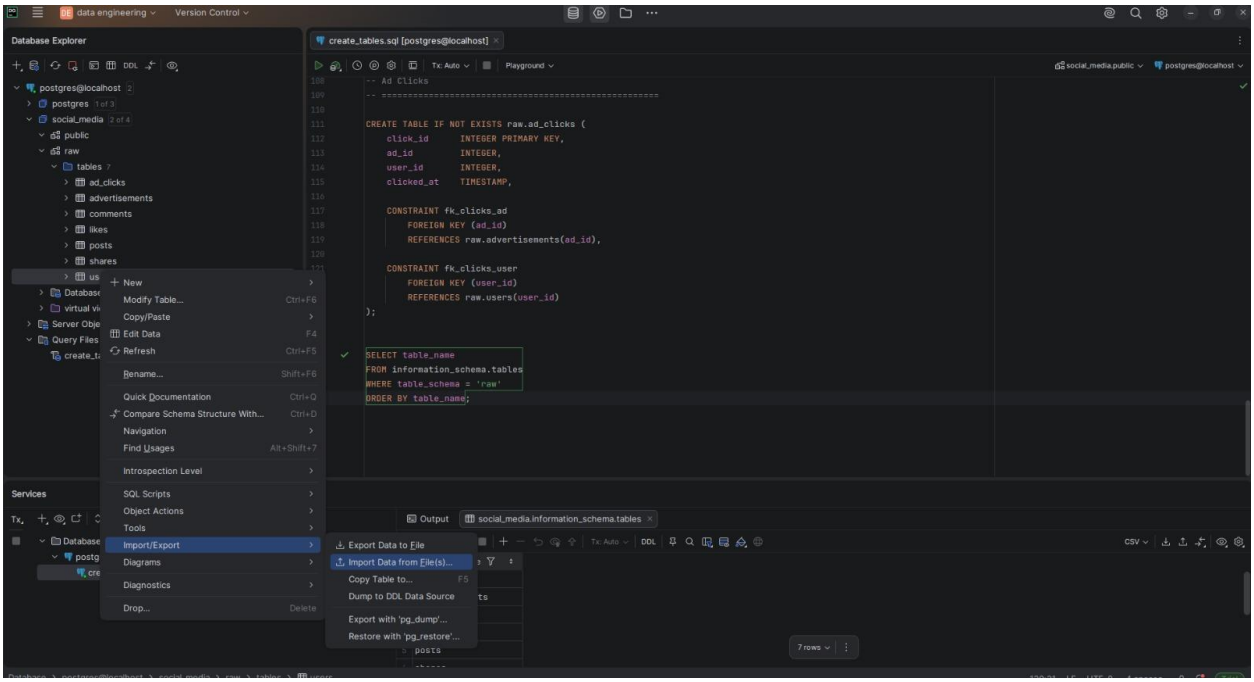
-- ====================================================
-- Ad Clicks
-- ====================================================

CREATE TABLE IF NOT EXISTS raw.ad_clicks (
    click_id      INTEGER PRIMARY KEY,
    ad_id         INTEGER,
    user_id       INTEGER,
    clicked_at    TIMESTAMP,

    CONSTRAINT fk_clicks_ad
        FOREIGN KEY (ad_id)
        REFERENCES raw.advertisements(ad_id),

    CONSTRAINT fk_clicks_user
        FOREIGN KEY (user_id)
        REFERENCES raw.users(user_id)
);
```

Insert data



Repeat for:

- users.csv -> raw.users
- posts.csv -> raw.posts
- comments.csv -> raw.comments
- likes.csv → raw.likes
- shares.csv → raw.shares
- advertisements.csv → raw.advertisements
- ad_clicks.csv → raw.ad_clicks

Basic SQL

Q1. Display all records from the `users` table.

Q2. Display only the username and email address of all users.

Q3. Find users who joined after January 1st, 2025.

Q4. Display the latest 10 posts.

Q5. Display all posts sorted from newest to oldest.

Data Quality Assessment

Q6. Find users with missing email addresses.

Q7. Find posts with empty or missing content.

Q8. Identify duplicate email addresses.

Q9. Find comments that reference posts that do not exist.

Q10. Generate a report showing the number of records in each table.

Aggregations

Q11. Count the total number of users.

Q12. Count the total number of posts.

Q13. Calculate the number of posts created by each user.

Q14. Find users who have created more than 5 posts.

Q15. Calculate the average number of posts per user.

JOIN Operations

Q16. Display users and the posts they created using an INNER JOIN.

Q17. Display all users and their posts using a LEFT JOIN.

Q18. Display all posts and their users using a RIGHT JOIN.

Q19. Display all matching and non-matching records using a FULL OUTER JOIN.

Q20. Find users who joined on the same date using a SELF JOIN.

Data Validation

Q21. Find users who have never created a post.

Q22. Find posts that have never received a comment.

Q23. Find posts that have never received a like.

Q24. Find advertisements that have never been clicked.

Q25. Find users who have never liked, commented, or shared any post.

Subqueries

Q26. Find all posts created by a user named 'john'.

Q27. Find users who have created at least one post using EXISTS.

Q28. Find users whose IDs appear in the posts table using IN.

Q29. Find users whose post count is greater than the average post count.

Q30. Find the most active user on the platform.

Common Table Expressions (CTE)

Q31. Create a CTE that calculates the number of posts per user.

Q32. Using a CTE, find users with above-average post counts.

Q33. Using multiple CTEs, compare post counts and like counts for each user.

Q34. Create a CTE to calculate engagement scores for each post.

Q35. Find the top 10 most engaging posts using a CTE.

Window Functions

Q36. Assign a row number to each user based on join date.

Q37. Rank users based on the number of posts they created.

Q38. Display the top 3 users by post count.

Q39. Calculate a dense rank for users based on post count.

Q40. Display each user's post count together with their ranking.

Views

Q41. Create a view showing the total number of posts created by each user.

Q42. Create a view showing the total number of comments received by each post.

Q43. Create a view showing the total number of likes received by each post.

Q44. Create a materialized view showing user activity statistics.

Q45. Refresh the materialized view after new data is loaded.

Reporting & Analytics

Q46. Generate a report showing the top 10 most active users.

Q47. Generate a report showing the top 10 most liked posts.

Q48. Generate a report showing the top 10 most commented posts.

Q49. Generate a monthly user growth report.

Q50. Create an executive KPI report showing total users, posts, comments, likes, shares, advertisements, and advertisement clicks.

Transactions

Q51. Create a transaction that inserts a new post and commits the changes.

Q52. Create a transaction that inserts a post and rolls back the changes.

Q53. Demonstrate the use of SAVEPOINT within a transaction.

Q54. Explain the ACID properties of transactions.

Q55. Demonstrate the READ COMMITTED isolation level.

Performance & Optimization

Q56. Use EXPLAIN ANALYZE to inspect a query filtering by user_id.

Q57. Create an index on user_id in the posts table.

Q58. Compare query performance before and after indexing.

Q59. Create a composite index using user_id and created_at.

Q60. Explain what happens internally when an INSERT occurs on an indexed table.

Advanced Data Engineering Concepts

Q61. Explain what happens internally when an UPDATE occurs on an indexed table.

Q62. Explain what happens internally when a DELETE occurs on an indexed table.

Q63. Run VACUUM ANALYZE on a table and explain its purpose.

Q64. Cluster a table using an index and explain the benefits.

Q65. Create a partitioned posts table based on created_at.

Q66. Create a monthly partition for January 2025.

Q67. Explain partition pruning.

Q68. Explain the purpose of the raw schema.

Q69. Explain the purpose of the sample_cde schema.

Q70. Explain the purpose of the analytics schema.